

MelodyMap

Aplicación Web para Análisis de Hábitos Musicales

Proyecto Final de Ciclo

2º Desarrollo de Aplicaciones Web (DAW)

David Pastor Berenguer

Tutor: Andrés/Jorge

CIP FP BATOI

Índice

1. INTRODUCCIÓN.....	5
1.1. Motivación del proyecto.....	5
1.2. Objetivos del proyecto.....	5
Objetivos específicos implementados:.....	5
1.3. Alcance del proyecto.....	5
2. ANÁLISIS Y DISEÑO DEL SISTEMA.....	7
2.1. Modelo de Datos.....	7
2.2. Relaciones entre Entidades.....	7
3. ARQUITECTURA Y TECNOLOGÍAS.....	8
3.1. Arquitectura General.....	8
3.2. Stack Backend.....	8
3.3. Stack Frontend.....	8
4. IMPLEMENTACIÓN DETALLADA.....	10
4.1. Sistema de Autenticación.....	10
4.2. Gestión de Artistas.....	10
4.3. Gestión de Canciones y Álbumes.....	11
4.4. Registro de Escuchas - Wizard Multi-Paso.....	11
4.5. Dashboard y Estadísticas.....	12
4.6. Sistema de Amigos.....	13
4.6.1. Búsqueda y Solicitudes.....	13
4.6.2. Gestión Solicitudes.....	13
4.6.3. Lista de Amigos.....	13
4.6.4. Estadísticas de Amigos.....	13
4.7. Sistema de Playlists Públicas.....	15
4.7.1. Visualización.....	15

4.7.2. Detalles.....	15
4.7.3. Permisos.....	15
4.7.4. Añadir Canciones.....	15
4.8. Sistema de Recomendaciones.....	16
4.8.1. Canciones Populares entre Amigos.....	16
4.8.2. Artistas a Descubrir.....	16
4.8.3. Playlists de Amigos.....	16
4.8.4. Estado Sin Amigos.....	16
5. API REST - DOCUMENTACIÓN.....	17
5.1. Resumen de Endpoints.....	17
5.2. Autenticación.....	17
6. CONCLUSIONES Y TRABAJO FUTURO.....	19
6.1. Objetivos Alcanzados.....	19
6.2. Tecnologías Aplicadas.....	19
6.3. Dificultades Superadas.....	19
6.4. Trabajo Futuro.....	20
6.5. Valoración personal.....	20
6.5. Conexión con las FCT.....	20

1. INTRODUCCIÓN

1.1. Motivación del proyecto

En la era digital actual, la música ha experimentado una transformación radical. Las plataformas de streaming han revolucionado la industria, pero existe una carencia significativa en aplicaciones que permitan analizar hábitos musicales de manera social y personalizada. MelodyMap nace como una plataforma completa donde los usuarios pueden registrar, visualizar, analizar y compartir sus escuchas musicales con amigos.

1.2. Objetivos del proyecto

Desarrollar una aplicación web completa que permita registrar y analizar hábitos de escucha musical con dashboards interactivos, además de crear una experiencia social.

Objetivos específicos implementados:

- Sistema de autenticación con Laravel Sanctum (tokens de acceso personal)
- CRUD completo de artistas con 13 campos de información
- CRUD de canciones con gestión dinámica de álbumes
- Wizard multi-paso para registrar escuchas
- Dashboard con gráficos interactivos (Recharts)
- Sistema de amigos completo (búsqueda, solicitudes, estadísticas)
- Playlists públicas visibles para todos
- Sistema de recomendaciones basado en amigos
- API REST con 35 endpoints

1.3. Alcance del proyecto

MelodyMap es una aplicación web full-stack completamente funcional con características core, sociales y de descubrimiento de contenido:

Funcionalidades Core:

- Autenticación completa (registro, login, logout)
- Gestión de artistas (13 campos + redes sociales)
- Gestión de canciones y álbumes
- Registro de escuchas con wizard de 4 pasos
- Dashboard con 3 tipos de gráficos + métricas en tiempo real

Funcionalidades Sociales:

- Búsqueda de usuarios por nombre/email
- Sistema de solicitudes de amistad (pendiente/aceptada)
- Vista de estadísticas completas de amigos
- Visualización de playlists creadas por amigos
- Sistema de recomendaciones inteligente

Funcionalidades de Contenido:

- Playlists públicas que todos pueden ver
- CRUD completo de playlists
- Exploración de artistas con canciones por álbum
- Creación dinámica de álbumes

2. ANÁLISIS Y DISEÑO DEL SISTEMA

2.1. Modelo de Datos

La base de datos sigue el modelo relacional con 7 tablas principales:

Tabla	Propósito	Campos Principales
users	Usuarios	name, email, password
artists	Artistas	name, bio, image, país, género, discográfica, redes sociales (13 campos)
songs	Canciones	title, duration, album, artist_id
listens	Escuchas	user_id, song_id, listened_at
playlists	Playlists	user_id, name, description
friendships	Amistades	user_id, friend_id, status (pending/accepted)
playlist_song	Pivot	playlist_id, song_id

2.2. Relaciones entre Entidades

1. User → Playlists (1:N)

Un usuario crea múltiples playlists. Las playlists son públicas pero solo el creador puede editarlas.

2. User → Listens (1:N)

Un usuario registra múltiples escuchas con timestamp.

3. Artist → Songs (1:N)

Un artista tiene múltiples canciones. Eliminación en cascada.

4. Playlist ↔ Song (N:M)

Relación many-to-many mediante tabla pivot playlist_song.

5. User ↔ User (Friendships N:M)

Relación auto-referencial con status (pending/accepted).

6. Song → Listens (1:N)

Una canción tiene múltiples escuchas. Base de las estadísticas.

3. ARQUITECTURA Y TECNOLOGÍAS

3.1. Arquitectura General

Arquitectura cliente-servidor con separación total entre frontend y backend, comunicándose mediante API REST sobre HTTP/HTTPS.

Componentes principales:

- Backend API REST: Laravel 12 con PHP 8.2
- Frontend: React 19.2 con React Router v6
- Base de datos: MySQL 8.0
- Autenticación: Laravel Sanctum (tokens de acceso personal)
- Comunicación: Axios con interceptores
- Visualización: Recharts
- Estilos: Tailwind CSS + CSS inline

3.2. Stack Backend

Laravel 12 (PHP 8.2)

Framework PHP con patrón MVC, Eloquent ORM, migraciones, validación y Laravel Sanctum.

MySQL 8.0

SGBD relacional con transacciones, índices compuestos y foreign keys.

Laravel Sanctum

Sistema de autenticación con tokens de acceso personal (NO JWT). Tokens aleatorios almacenados hasheados en personal_access_tokens.

3.3. Stack Frontend

React 19.2

Biblioteca para UI con componentes reutilizables y hooks (useState, useEffect, useNavigate).

React Router v6

Enrutamiento con rutas protegidas (ProtectedRoute).

Axios

Cliente HTTP con interceptores para token Bearer automático y manejo de errores 401.

Recharts

Gráficos interactivos: BarChart, LineChart con ejes configurables.

4. IMPLEMENTACIÓN DETALLADA

4.1. Sistema de Autenticación

Laravel Sanctum con tokens de acceso personal. Tokens aleatorios hasheados en BD.

Flujo de autenticación:

- Usuario envía credenciales a POST /api/login
- Backend valida con Hash::check()
- Si válido, genera token con \$user->createToken()
- Token hasheado en personal_access_tokens
- Frontend guarda en localStorage
- Axios incluye Bearer {token} automáticamente
- Middleware auth:sanctum valida en cada petición

Endpoints:

- POST /api/register - Registro con email único
- POST /api/login - Login con generación de token
- POST /api/logout - Invalidación del token
- GET /api/profile - Datos del usuario autenticado

4.2. Gestión de Artistas

CRUD completo con 13 campos de información detallada incluyendo redes sociales.

Campos del modelo Artist:

- name (requerido) - Nombre del artista
- bio (opcional) - Biografía
- image (opcional) - URL imagen
- pais (requerido) - País de origen
- genero (requerido) - Solista/Banda/Dúo/Grupo
- genero_musical (requerido) - Género musical
- fecha_de_nacimiento (requerido) - Fecha
- discográfica (opcional)
- youtube (requerido) - URL
- spotify (opcional) - URL
- instagram (opcional) - Usuario
- other_links (opcional) - Otros enlaces

Funcionalidades UI:

- Listado con tarjetas compactas
- Búsqueda en tiempo real
- Canciones agrupadas por álbum

- Modal de creación con validación de 13 campos
- Creación de canciones con selector dinámico

4.3. Gestión de Canciones y Álbumes

Canciones vinculadas a artistas con organización por álbumes. Creación dinámica de álbumes.

Flujo de creación:

- Seleccionar artista existente
- Click "Nueva Canción" abre modal con: título, duración (preview mm:ss), álbum
- Selector de álbum: Sin álbum / Existentes / "+ Escribir nuevo álbum..."
- Al seleccionar nuevo álbum, input dinámico aparece
- Submit crea canción + álbum si es nuevo

4.4. Registro de Escuchas - Wizard Multi-Paso

Wizard de 4 pasos validado para registrar escuchas intuitivamente.

Paso 1: Selección de Artista

- Búsqueda en tiempo real
- Tarjetas con avatar
- Click para seleccionar y avanzar

Paso 2: Selección de Canción

- Solo canciones del artista
- Búsqueda por título
- Info: título, álbum, duración
- Botón crear canción nueva

Paso 3: Fecha y Hora

- Input datetime-local
- Por defecto: ahora (timezone Europe/Madrid)
- Validación: no futuro

Paso 4: Confirmación

- Resumen visual completo
- Artista + canción + fecha
- Botón confirmar → API

4.5. Dashboard y Estadísticas

3 gráficos interactivos con Recharts, filtros por fechas y métricas en tiempo real.

Métricas principales:

- Total escuchas - COUNT(*) listens
- Canciones únicas - COUNT(DISTINCT song_id)
- Artistas únicos - COUNT(DISTINCT artist_id) con JOIN

Gráfico 1: Top 10 Canciones (BarChart horizontal)

- GROUP BY song_id ORDER BY COUNT(*) DESC LIMIT 10
- Eje Y: Títulos, Eje X: Reproducciones
- Color verde #2d5016

Gráfico 2: Top 10 Artistas (BarChart vertical)

- JOIN songs → GROUP BY artist_id
- Eje X: Nombres (rotados -45°)
- Color verde medio #7d9d6f

Gráfico 3: Evolución Mensual (LineChart)

- GROUP BY YEAR, MONTH últimos 12 meses
- Eje X: Meses (Ene 2025)
- Línea continua con puntos

Sistema de filtros:

- Rango fechas (desde - hasta)
- Recálculo automático todas las métricas

4.6. Sistema de Amigos

Sistema social completo para conectar con amigos, ver estadísticas y descubrir música.

4.6.1. Búsqueda y Solicitudes

- Búsqueda usuarios por nombre/email (min 2 caracteres)
- Resultados con nombre, email, estado amistad
- Botón "Enviar Solicitud" → status=pending
- Validación: no a uno mismo, no duplicados

4.6.2. Gestión Solicitudes

- Vista solicitudes recibidas
- Info solicitante: avatar, nombre, email
- Botones Aceptar (status=accepted) y Rechazar (elimina)
- Badge contador pendientes

4.6.3. Lista de Amigos

- Amigos en ambas direcciones
- Tarjetas avatar + info
- Botón "Ver Estadísticas" → /friends/{id}/stats
- Botón "Eliminar" con confirmación

4.6.4. Estadísticas de Amigos

Vista completa de hábitos musicales del amigo:

4 métricas principales:

- Total escuchas
- Canciones únicas
- Artistas únicos
- Playlists creadas

Favoritos:

- Canción más escuchada con contador
- Artista favorito con reproducciones

Gráficos:

- Top 10 canciones (BarChart)
- Top 10 artistas (BarChart)

Playlists:

- Grid responsive de playlists
- Info: nombre, descripción, número canciones, fecha

- Cards clickables → vista playlists

4.7. Sistema de Playlists Públicas

Playlists visibles para todos, editables solo por creador. Fomenta descubrimiento musical.

4.7.1. Visualización

- Listado completo TODAS las playlists
- Info: nombre, descripción, creador, # canciones
- Buscador por nombre
- Layout 2 columnas: lista + detalles
- Indicador "  {creador}"

4.7.2. Detalles

- Header nombre completo + descripción
- Texto "Creada por: {nombre}"
- Estadísticas: # canciones + duración total
- Lista numerada: título, artista, álbum, duración

4.7.3. Permisos

Controles solo si `currentUser.id === playlist.user_id`:

- Botón  Editar - Modal edición
- Botón  Eliminar - Con confirmación
- Botón  Añadir Canciones - Modal selección
- Botón  en canciones - Quitar (solo creador)

4.7.4. Añadir Canciones

- Modal con TODAS las canciones sistema
- Filtro automático: oculta ya en playlist
- Búsqueda por título o artista
- Botón  Añadir por canción
- Detección duplicados: error 409
- Actualización tiempo real contador

4.8. Sistema de Recomendaciones

Análisis en tiempo real de hábitos de amigos mediante queries SQL hechas con Eloquent.

4.8.1. Canciones Populares entre Amigos

Query: canciones que amigos escucharon pero usuario NO.

Lógica:

- Obtener IDs amigos (status=accepted)
- SELECT listens WHERE user_id IN (friend_ids)
- Filtrar: song_id NOT IN (canciones usuario)
- GROUP BY song_id con: COUNT(DISTINCT user_id), COUNT(*)
- ORDER BY friend_count DESC, total_listens DESC
- LIMIT 10

Información mostrada:

- Título, artista, álbum, duración
- "X amigos la escucharon (Y veces)"
- Botón "Registrar Escucha"

4.8.2. Artistas a Descubrir

- Artistas que amigos escuchan pero usuario NO
- JOIN listens → songs → artists
- Filtro: artist_id NOT IN (artistas usuario)
- Ordenados por popularidad
- LIMIT 8, Grid responsive
- Cards avatar 80px + género + badge "👥 X amigos"

4.8.3. Playlists de Amigos

- Playlists creadas por amigos
- Filtro: user_id IN (friend_ids) AND songs_count > 0
- Ordenadas por fecha (recientes primero)
- LIMIT 6, Cards con "Por {creador}"

4.8.4. Estado Sin Amigos

- Mensaje "¡Añade amigos para recomendaciones!"
- Botón "Ir a Amigos"

5. API REST - DOCUMENTACIÓN

API REST con 35+ endpoints siguiendo principios RESTful. Todos (excepto login/register) requieren token Bearer.

5.1. Resumen de Endpoints

Método	Endpoint	Descripción
POST	/api/register	Registro usuario
POST	/api/login	Login, retorna token
POST	/api/logout	Invalida token
GET	/api/profile	Datos usuario autenticado
GET	/api/artists	Listado artistas
GET	/api/artists/get/{id}	Obtener artista
POST	/api/artists/create	Crear artista
PATCH	/api/artists/update/{id}	Actualizar artista
DELETE	/api/artists/delete/{id}	Eliminar artista
GET	/api/songs	Listado canciones. ?artist_id=
GET	/api/songs/get/{id}	Obtener canción
POST	/api/songs/create	Crear canción
PATCH	/api/songs/update/{id}	Actualizar canción
DELETE	/api/songs/delete/{id}	Eliminar canción
GET	/api/listens	Historial escuchas usuario
POST	/api/listens/create	Registrar escucha
GET	/api/listens/statistics	Estadísticas. ?from=&to=
GET	/api/listens/{id}	Obtener escucha
DELETE	/api/listens/{id}	Eliminar escucha
GET	/api/playlists	TODAS las playlists
POST	/api/playlists/create	Crear playlist
GET	/api/playlists/{id}	Obtener playlist
PATCH	/api/playlists/{id}	Actualizar playlist
DELETE	/api/playlists/{id}	Eliminar playlist
POST	/api/playlists/{id}/songs	Añadir canción
DELETE	/api/playlists/{id}/songs	Quitar canción
GET	/api/friends/search	Buscar usuarios. ?query=
POST	/api/friends/send-request	Enviar solicitud
GET	/api/friends/pending	Solicitudes recibidas
POST	/api/friends/accept/{id}	Aceptar solicitud
DELETE	/api/friends/reject/{id}	Rechazar solicitud
GET	/api/friends	Lista amigos aceptados
DELETE	/api/friends/remove/{id}	Eliminar amistad
GET	/api/friends/{id}/stats	Estadísticas amigo
GET	/api/recommendations	Recomendaciones personalizadas

5.2. Autenticación

Laravel Sanctum con tokens de acceso personal. Token en header: Authorization: Bearer {token}

6. CONCLUSIONES Y TRABAJO FUTURO

6.1. Objetivos Alcanzados

MelodyMap es una aplicación web funcional que cumple los objetivos planteados:

- Sistema completo de autenticación seguro con Laravel Sanctum
- CRUD completo de artistas con 13 campos de información
- Gestión de canciones y álbumes con creación dinámica
- Wizard intuitivo de 4 pasos para registro de escuchas
- Dashboard con 3 tipos de gráficos interactivos
- Sistema de amigos completo con búsqueda, solicitudes y estadísticas
- Playlists públicas con permisos granulares
- Sistema de recomendaciones basado en análisis de hábitos
- API REST con 35+ endpoints documentados
- Frontend responsive con React 19.2
- Base de datos normalizada con 7 tablas y sus relaciones.

6.2. Tecnologías Aplicadas

El proyecto integra tecnologías modernas de desarrollo web:

Backend:

- Laravel 12 con Eloquent
- MySQL 8.0
- Laravel Sanctum para autenticación

Frontend:

- React 19.2 con hooks
- React Router v6 con rutas protegidas
- Axios con interceptores
- Recharts para visualización
- Tailwind CSS + CSS inline

6.3. Dificultades Superadas

Autenticación token Bearer:

Implementación de interceptores Axios para incluir token automáticamente. Inicialmente había errores 401 por falta de header Authorization.

Queries complejas para recomendaciones:

Diseño de queries SQL con múltiples JOINS, subqueries y agregaciones para identificar canciones/artistas que amigos escuchan pero usuario no.

Gestión de estado en wizards:

Sincronización de estado entre pasos y validación antes de avanzar. Implementado con useState y validaciones condicionales.

Playlists públicas con permisos:

Balance entre visibilidad pública y permisos de edición. Resuelto con verificación de user_id en backend y condicionales en UI.

Agrupación de canciones por álbum:

Organización visual de canciones por álbum con contador. Implementado con función groupSongsByAlbum() que procesa array y agrupa.

6.4. Trabajo Futuro

Posibles mejoras y expansiones:

- Integración con Spotify API para importar escuchas automáticamente
- Integración de DBT para modelado de datos
- Integración de Metabase para un excelente análisis y visualización de datos
- Sistema de notificaciones en tiempo real (nuevas solicitudes, playlists)
- Playlists colaborativas (múltiples usuarios pueden añadir canciones)
- Sistema de comentarios en playlists y canciones
- Análisis temporal avanzado (evolución gustos por mes/año)
- Exportación de estadísticas a PDF
- Modo oscuro
- Sistema de badges/logros por escuchas
- Comparativas "Tú vs Amigos"
- Feed de actividad en tiempo real

6.5. Valoración personal

Considero que este proyecto se ha hecho bastante pesado debido al poco tiempo que se le ha podido dedicar, pudiendo haber añadido las funcionalidades que me hubiese gustado desde un principio como DBT o Metabase.

6.5. Conexión con las FCT

En las FCT descubrí DBT y Metabase, tecnologías que me hubiese gustado añadir debido al alto manejo y visualización de datos que estas permiten.